0. **Code:** vemuriu6191 vemuriu6191.zip

1. large running averages/large running average/Program.cs
2. large running averages/nested loop/Program.cs

1. large running averages/large running average/Program.cs

```
namespace large_running_average
{
    internal class Program
    {
        public static void Main(string[] args)
        {
            int[] array = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };
            double average = CalculateAverage(array);
            Console.WriteLine("The average of the values in array1 that exceed their previously running averages is: " + average);
        }
        public static double CalculateAverage(int[] array)
        {
            double sum = 0;
            int count = 0;
            double previousSum = 0;
            double runningAverage = 0;
            for (int i = 0; i < array.Length; i++)
            {
                previousSum += array[i];
                runningAverage = previousSum / (i + 1);
```

A1

A3

if array == null || array.Length == 0) return

set to the previous

[i-1]

if(i > 0) {

after the first element, you have a previous,

need to check i>0. OK

1. **large running averages/large running average/Program.cs**

A20

```
                if (array[i] > runningAverage)
                {
                    sum += array[i];
                    count++;
                }
            }
            if (count == 0)
            {
                return 0;
            }
            else
            {
                return sum / count;
            }
        }
    }
}
```

A2

A3

2. large running averages/nested loop/Program.cs

```
namespace nested_loop
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] arr = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };
            int[] newArray = GetnewArray(arr);
            Console.WriteLine(string.Join(", ", newArray));
        }
        static int[] GetnewArray(int[] arr)
        {
            if (arr == null)
            {
                return null;
            }
            if (arr.Length < 2)
            {
                return arr;
            }
            int length = arr.Length * (arr.Length + 1) / 2;
```

B1

2. large running averages/nested loop/Program.cs

```
            int[] newArray = new int[length];
            int inserted = 0;
            for (int i = 0; i < arr.Length; i++)
            {
                Array.Copy(arr, newArray, arr.Length);
                for (int j = 0; j < newArray.Length - 1; j++)
                {
                    if (newArray[j] == newArray[j + 1] && newArray[j] ==
                      arr[i])
                    {
                        continue;
                    }
                    newArray[inserted] = newArray[j];
                    if (inserted > 0 && newArray[inserted - 1] ==
                      newArray[inserted])
                    {
                        j--;
                    }
                    inserted++;
                }
                newArray[inserted] = newArray[newArray.Length - 1];
                inserted++;
            }
            return newArray;
        }

    }
}
```

B2 goes outside the loop Use a for loop for this.

$O(n)$

$O(n^2)$

$O(n^3)$

B4 Did not attempt to Find the min!

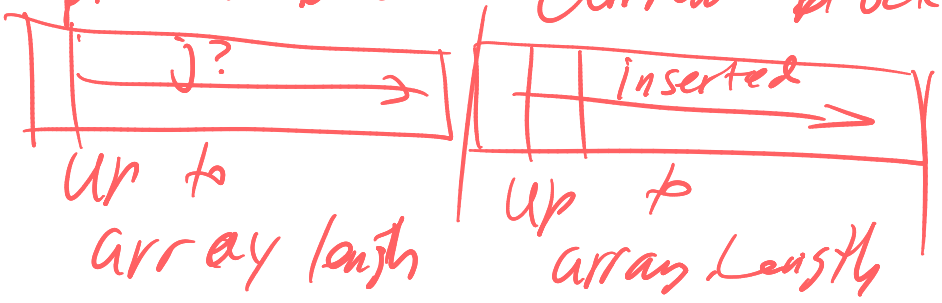
What? means when adjacent values are the same

Why? j is not the Index of newArray

B3 Irrelevant

B5

previous block



current block.

up to array length

up to array Length

$(i-1)^{th}$ iteration

i^{th} iteration.